

Hashing

Can We Search in Constant Time?

Data structure	Search	Insert	Delete
Unsorted array	$O(n)$	$O(1)$	$O(n)$
Sorted array	$O(\log n)$	$O(n)$	$O(n)$
Balanced search tree	$O(\log n)$	$O(\log n)$	$O(\log n)$
Array indexed by key	$O(1)$	$O(1)$	$O(1)$
Hash table	$O(1)$ average	$O(1)$ average	$O(1)$ average

Question: Can we look up **any** key in $O(1)$ time while using space proportional to the number of keys stored?
keys stored?

The Idea: Labelled Drawers

- Keep many small collections, called **buckets**, like the like the drawers of a dresser.
- A rule says which drawer each item belongs in.
- To find an item, open **only its drawer**.
- The rule is a **hash function**: $\text{bucket} = \text{hash}(\text{key}) \bmod \text{number of buckets}$.

bucket 0	ceres
bucket 1	
bucket 2	urania
bucket 3	vesta
bucket 4	
bucket 5	juno

Example: $\text{hash}(\text{"urania"}) = 12206$, and $12206 \bmod 6 = 2$, so “urania” goes in bucket 2.

What Is a Hash Function?

A function that maps **any input** (a number, a string, a file) to a value in a **fixed range**, usually an integer.

Deterministic. the same input always gives the same output.

Spreads inputs out. different inputs usually give different outputs, even when the inputs are very similar.

Input	name_hash(input)
"Kylie" + "J"	13172495
"Kylie" + "Jue"	127380

A Hash Function in Code

```
unsigned name_hash(const char *first, const char *last) {  
    const unsigned kLargePrime = 16908799;  
    const unsigned kSmallPrime = 127;  
    unsigned h = 0;  
    for (const char *s = first; *s; s++)           // letters of first name  
        h = (kSmallPrime * h + tolower(*s)) % kLargePrime;  
    for (const char *s = last; *s; s++)           // then of last name  
        h = (kSmallPrime * h + tolower(*s)) % kLargePrime;  
    return h;                                       // 0 .. kLargePrime - 1  
}
```

Each letter is mixed in by **multiplying by a prime and adding**; taking mod keeps the value in range.

Hash Tables Are Everywhere

- **Programming languages:** Python's dict and set, Java's HashMap, C++'s unordered_map.
- **Compilers:** the symbol table maps each identifier to its declaration.
- **Databases:** hash indexes find records by key; hash joins match rows of two tables.
- **Caches:** web caches and memoised functions look results up by key.
- **Spell checkers and dictionaries:** is this word in the dictionary?

Hash Functions Beyond Tables

- **Passwords**: systems store the hash of a password, not the password, so a stolen database does not reveal it.
- **Integrity**: comparing hashes shows whether a file or message was changed in transit.
- **Fingerprints**: songs and documents are recognised from hashes of small pieces.
- **Finding repeats**: duplicate files, copied text and common substrings are found by comparing hashes instead of whole strings.

The Dictionary Problem

A **dynamic set** that supports only three operations:

INSERT(T, x): add element x

SEARCH(T, k): find the element with key k

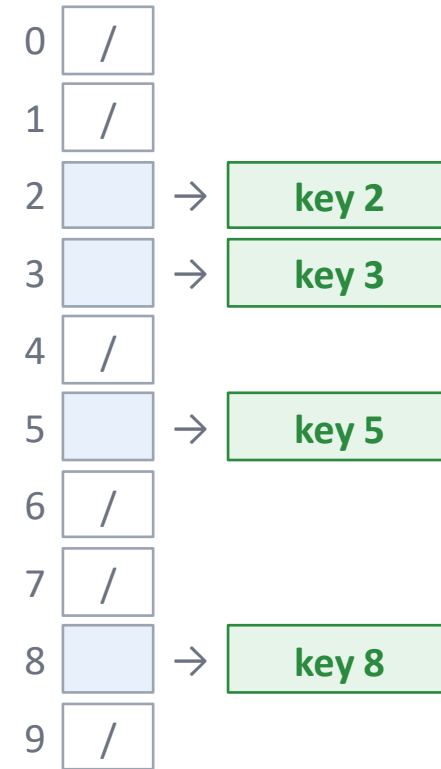
DELETE(T, x): remove element x

Examples: a compiler's **symbol table**, a phone book, a cache, Python's *dict*, Java's *HashMap*.

Goal. all three operations in **$O(1)$** average time.

Direct-Address Tables

- Keys come from $U = \{0, \dots, 9\}$; stored keys are 2, 3, 5, 8.
- Slot k holds the element with key k ; empty slots hold **NIL** (drawn $/$).
- Every operation is **$O(1)$** , worst case.



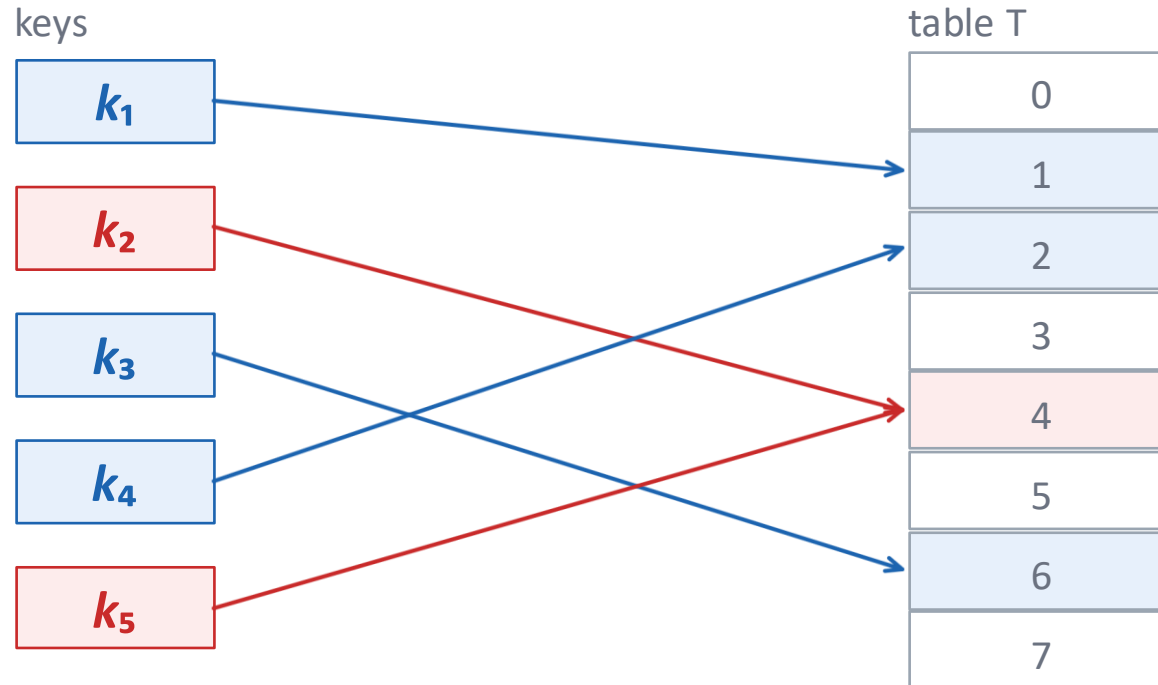
When the Universe Is Too Large

Problem. a direct-address table needs a slot for **every possible key**. For 64-bit keys that is about 1.8×10^{19} slots.

Observation. we usually store few keys, e.g. **10,000 students** with 64-bit IDs. Almost every slot would be empty.

Use a table with only m slots and compute the slot from the key with a **hash function** h .

Hash Tables



- A **hash function** h sends each key to a slot $0 \dots m - 1$.
- Key k is stored in slot $h(k)$.
- k_2 and k_5 both go to slot 4: a **collision**.

Collisions Are Unavoidable

Pigeonhole principle. there are more possible keys than slots, so some keys must share a slot.

Birthday paradox. with just **23 people**, the chance that two share a birthday is **50.7%**. Collisions appear long before the table is full.

Two Ways to Resolve Collisions

Chaining. each slot holds a **linked list** of all keys that hash there.

Open addressing. all keys live **in the table itself**; on a collision, try another slot.

Open Addressing

Open Addressing

- Every slot holds **at most one key**.
- To insert, **probe** slots in a fixed order until one is empty.
- Search follows the **same order**.

0	1	2	3	4	5	6	7	8	9	10
22	88			4	15	28	17		31	10

Inserting **59**: start at $59 \bmod 11 = 4$, then 5, 6, 7, 8 (probe numbers on top).

				1	2	3	4	5		
0	1	2	3	4	5	6	7	8	9	10
22	88			4	15	28	17	59	31	10

HASH-INSERT

```
HASH-INSERT(T, k)
```

```
  i = 0
```

```
  repeat
```

```
    j = h(k, i)
```

```
    if T[j] == NIL
```

```
      T[j] = k
```

```
      return j
```

```
    else i = i + 1
```

```
until i == m
```

```
error "hash table overflow"
```

Try slots $h(k, 0)$, $h(k, 1)$, ... until an empty one is found.

HASH-SEARCH

```
HASH-SEARCH(T, k)
```

```
i = 0
```

```
repeat
```

```
    j = h(k, i)
```

```
    if T[j] == k
```

```
        return j
```

```
    i = i + 1
```

```
until T[j] == NIL or i == m
```

```
return NIL
```

An **empty slot** ends the search: the key would have been placed there.

Probe Sequences

With open addressing, each key k has a **probe sequence**: the order in which slots are tried.

Requirement. for every key, $h(k, 0), h(k, 1), \dots, h(k, m - 1)$ must visit **every slot exactly once** (a permutation of $0, \dots, m - 1$). Otherwise an insert can fail while empty slots remain.

Ideal (uniform hashing). each key's probe sequence is equally likely to be any of the $m!$ permutations. Real schemes approximate this.

Three Probe Sequences

Linear probing. try the next slot, then the next, ... Only m different probe sequences.

$$h(k, i) = (h_0(k) + i) \bmod m$$

Quadratic probing. jumps grow quadratically. Still only m different probe sequences.

$$h(k, i) = (h_0(k) + c_1 i + c_2 i^2) \bmod m, \quad c_1, c_2 \not\equiv 0 \pmod{m}$$

Double hashing. a second hash function sets the step size. About m^2 different probe sequences.

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m, \quad \gcd(h_2(k), m) = 1$$

Linear Probing

$$h(k, i) = (h_0(k) + i) \bmod m$$

- Start at the home slot $h_0(k)$ and step by **+1**, wrapping around at the end.
- **Simple and fast**: consecutive probes touch neighbouring memory.
- The first probe decides the whole sequence, so there are only m different sequences.
- Problem: occupied slots form long runs, called **primary clustering**.

Linear Probing, Step by Step (1/3)

$$h(k) = k \% 11$$

insert 10: $h'(10) = 10$

probes 10

no collision

slot 0 1 2 3 4 5 6 7 8 9 10



insert 22: $h'(22) = 0$

probes 0

no collision



insert 31: $h'(31) = 9$

probes 9

no collision



orange: slot where the key lands · **red border:** occupied slot probed on the way · **blue:** earlier keys

Linear Probing, Step by Step (2/3)

insert 4: $h'(4) = 4$

probes 4

no collision

slot	0	1	2	3	4	5	6	7	8	9	10
	22				4					31	10

insert 15: $h'(15) = 4$

probes 4 → 5

1 collision

	22				4	15				31	10
--	----	--	--	--	---	----	--	--	--	----	----

insert 28: $h'(28) = 6$

probes 6

no collision

	22				4	15	28			31	10
--	----	--	--	--	---	----	----	--	--	----	----

Linear Probing, Step by Step (3/3)

insert 17: $h'(17) = 6$

probes $6 \rightarrow 7$

1 collision

slot	0	1	2	3	4	5	6	7	8	9	10
	22				4	15	28	17		31	10

insert 88: $h'(88) = 0$

probes $0 \rightarrow 1$

1 collision

22	88			4	15	28	17		31	10
----	----	--	--	---	----	----	----	--	----	----

insert 59: $h'(59) = 4$

probes $4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$

4 collisions

22	88			4	15	28	17	59	31	10
----	----	--	--	---	----	----	----	----	----	----

A Cluster Forms: Linear Probing

Insert 3, 14, 25, 36, 47 (home slot 3), then 4, 15 (home slot 4); $m = 11$. Each row: key, probes.

slot	0	1	2	3	4	5	6	7	8	9	10
3 : 3				3							
14 : 3 → 4				3	14						
25 : 3 → 4 → 5				3	14	25					
36 : 3 → 4 → 5 → 6				3	14	25	36				
47 : 3 → 4 → 5 → 6 → 7				3	14	25	36	47			
4 : 4 → 5 → 6 → 7 → 8				3	14	25	36	47	4		
15 : 4 → 5 → 6 → 7 → 8 → 9				3	14	25	36	47	4	15	

Total. 26 probes for 7 keys. Keys 4 and 15 never collided with each other at home, yet paid 5 and 6 probes.

Primary Clustering

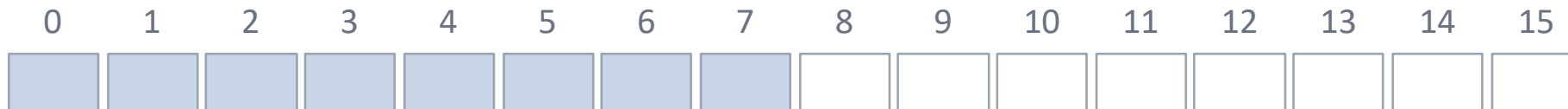
Primary clustering. long runs of occupied slots build up. Any key that hashes into a run must walk to its end, and then makes the run longer.

Two tables with $m = 16$ and 8 keys each (shaded = full/occupied):

Spread out: every other slot full



One run: slots 0–7 full



In the second table the next key extends the run with probability **9/16**: any home slot 0–8. An empty slot preceded by i full slots is filled next with probability $(i + 1)/m$.

Primary Clustering: Cost of a Failed Search

Average probes for an unsuccessful search, starting from a random slot (counting the final empty slot):

$$\text{every other slot full: } \frac{1}{m} \left[\frac{m}{2} \cdot 1 + \frac{m}{2} \cdot 2 \right] = 1.5$$

$$\text{one run of } m/2 \text{ full slots: } \frac{1}{m} \left[\frac{m}{2} \cdot 1 + \sum_{i=1}^{m/2} (i + 1) \right] = \frac{m}{8} + \frac{5}{4}$$

Same load, very different cost. both tables are half full ($\alpha = 1/2$), but with one run the cost **grows with m**: for $m = 16$ it is 3.25 probes versus 1.5, and for $m = 1000$ about 126 versus 1.5.

Quadratic Probing

$$h(k, i) = (h_0(k) + c_1i + c_2i^2) \bmod m, \quad c_1, c_2 \not\equiv 0 \pmod{m}$$

- Offsets grow as $c_1i + c_2i^2$, so a colliding key **jumps away** from the run instead of walking along it.
- This avoids primary clustering.
- c_1, c_2 and m must be chosen so that every slot is visited
- Keys with the same home slot still share the whole sequence: **secondary clustering**.

Secondary Clustering

Secondary clustering. keys with the same home slot follow the **same probe sequence**, so they collide with each other again and again. It is milder than primary clustering.

First five probes for three keys with home slot 3 ($m = 11$), with $h_0(k) = k \% 11$:

Key	Home	Quadratic probing ($c_1 = c_2 = 1$)	Double hashing (step $1 + k \bmod 10$)
3	3	$3 \rightarrow 5 \rightarrow 9 \rightarrow 4 \rightarrow 1$	$3 \rightarrow 7 \rightarrow 0 \rightarrow 4 \rightarrow 8$
14	3	$3 \rightarrow 5 \rightarrow 9 \rightarrow 4 \rightarrow 1$	$3 \rightarrow 8 \rightarrow 2 \rightarrow 7 \rightarrow 1$
25	3	$3 \rightarrow 5 \rightarrow 9 \rightarrow 4 \rightarrow 1$	$3 \rightarrow 9 \rightarrow 4 \rightarrow 10 \rightarrow 5$

Remedy. double hashing: the step size depends on the key, so keys with the same home slot spread out.

Double Hashing

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m, \quad \gcd(h_2(k), m) = 1$$

- h_1 picks the home slot; h_2 picks the step size, so different keys take different paths.
- Two standard choices: m a power of 2 and h_2 always odd; or m prime and $1 \leq h_2(k) < m$.
- About m^2 different probe sequences: **close to ideal** in practice.

Same Keys: Double Hashing

Insert 3, 14, 25, 36, 47 (home slot 3), then 4, 15 (home slot 4); $m = 11$. Each row: key, probes.

slot	0	1	2	3	4	5	6	7	8	9	10
3 (step 4): 3				3							
14 (step 5): 3 → 8				3					14		
25 (step 6): 3 → 9				3					14	25	
36 (step 7): 3 → 10				3					14	25	36
47 (step 8): 3 → 0	47			3					14	25	36
4 (step 5): 4	47			3	4				14	25	36
15 (step 6): 4 → 10 → 5	47			3	4	15			14	25	36

Total. 13 probes for 7 keys. Compare: 26 with linear probing, 20 with quadratic probing.

Deletion in Open Addressing

Delete **15** (slot 5), then search for **59**: probes 4, 5, 6, 7, 8.

slot

0 1 2 3 4 5 6 7 8 9 10

Wrong: empty slot 5.

Search stops at 5: **not found**

22	88			4		28	17	59	31	10
----	----	--	--	---	--	----	----	----	----	----

Right: mark it DEL.

Search continues: **found**

22	88			4	DEL	28	17	59	31	10
----	----	--	--	---	-----	----	----	----	----	----

Rule. store DELETED instead of NIL. Search skips it; insert may reuse it.

Proof: Unsuccessful Search with Open Addressing

Theorem. with uniform hashing and $\alpha = n/m < 1$, an unsuccessful search makes at most $1/(1 - \alpha)$ probes on average.

Let X be the number of probes. $X \geq i$ exactly when the first $i - 1$ probes all hit full slots. The first hits one of n full slots among m ; the next one of $n - 1$ among the remaining $m - 1$; and so on:

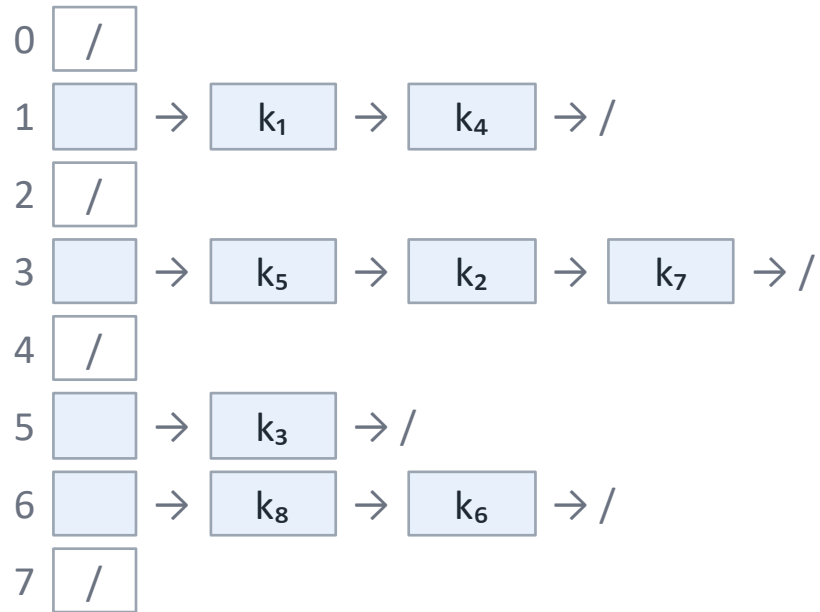
$$\Pr[X \geq i] = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdots \frac{n-i+2}{m-i+2} \leq \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1}$$

$$\mathbb{E}[X] = \sum_{i=1}^{\infty} \Pr[X \geq i] \leq \sum_{i=1}^{\infty} \alpha^{i-1} = 1 + \alpha + \alpha^2 + \cdots = \frac{1}{1 - \alpha}$$

Insertion is an unsuccessful search followed by one write, so it has the same bound. ■

Chaining

Chaining



- Keys that hash to the same slot form a **linked list**.
- Slot 3 holds three keys: k_5, k_2, k_7 .
- Empty slots hold NIL (/).

Operations with Chaining

Operation	What it does	Time
INSERT(T, x)	insert x at the head of list $T[h(x.key)]$	$O(1)$
SEARCH(T, k)	search for key k in list $T[h(k)]$	length of list
DELETE(T, x)	remove x from list $T[h(x.key)]$	$O(1)$

Note. DELETE is $O(1)$ with a **doubly linked** list, given a pointer to x .

Chaining, Step by Step (1/3)

$m = 9, h(k) = k \bmod 9$

insert 5: $5 \bmod 9 = 5$

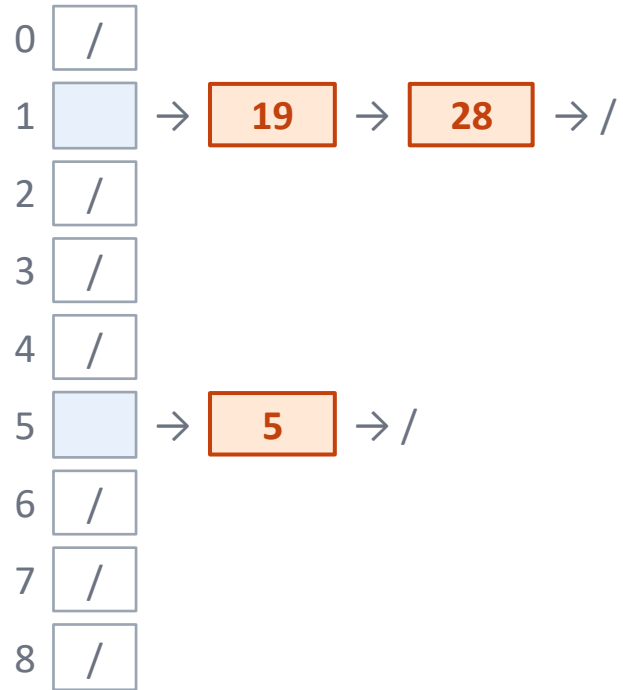
list 5 was **empty**

insert 28: $28 \bmod 9 = 1$

list 1 was **empty**

insert 19: $19 \bmod 9 = 1$

collision: goes to head of list 1



Chaining, Step by Step (2/3)

$m = 9, h(k) = k \bmod 9$

insert 15: $15 \bmod 9 = 6$

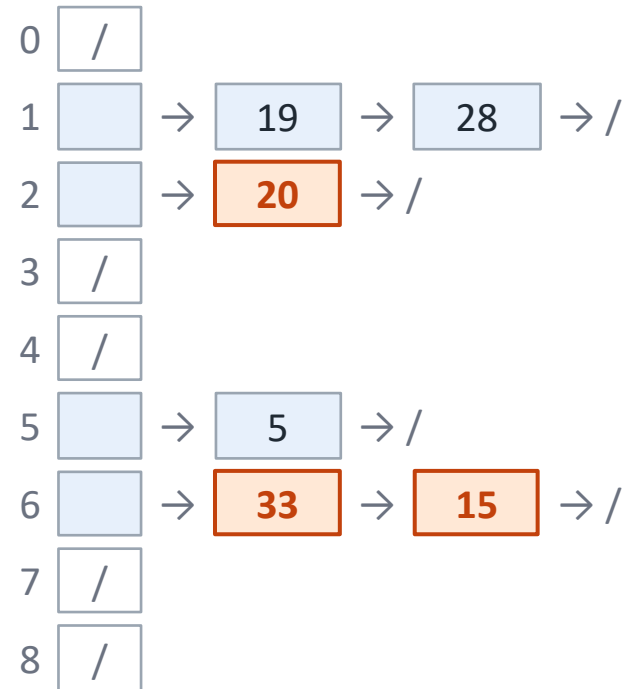
list 6 was **empty**

insert 20: $20 \bmod 9 = 2$

list 2 was **empty**

insert 33: $33 \bmod 9 = 6$

collision: goes to head of list 6



Chaining, Step by Step (3/3)

$m = 9, h(k) = k \bmod 9$

insert 12: $12 \bmod 9 = 3$

list 3 was **empty**

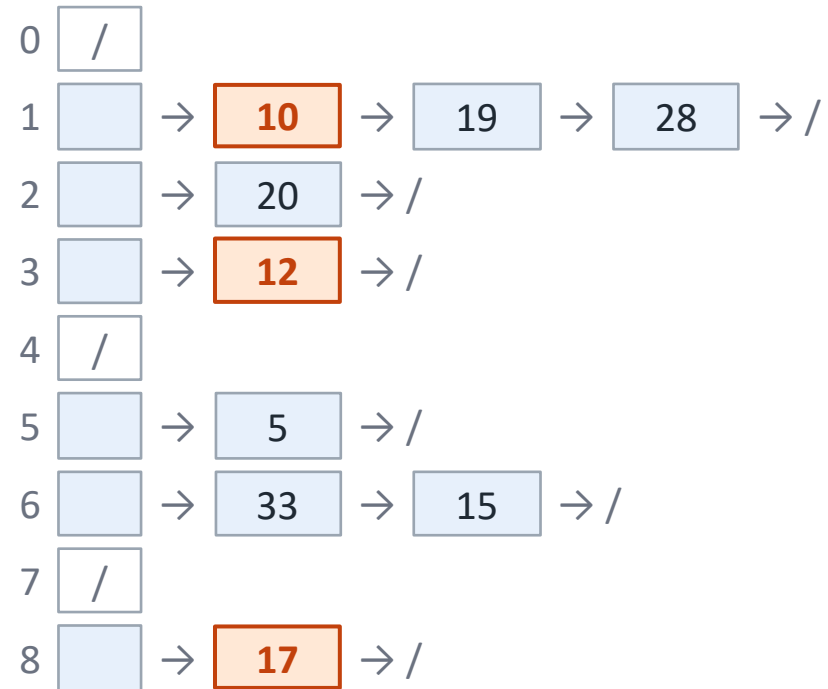
insert 17: $17 \bmod 9 = 8$

list 8 was **empty**

insert 10: $10 \bmod 9 = 1$

collision: goes to head of list 1

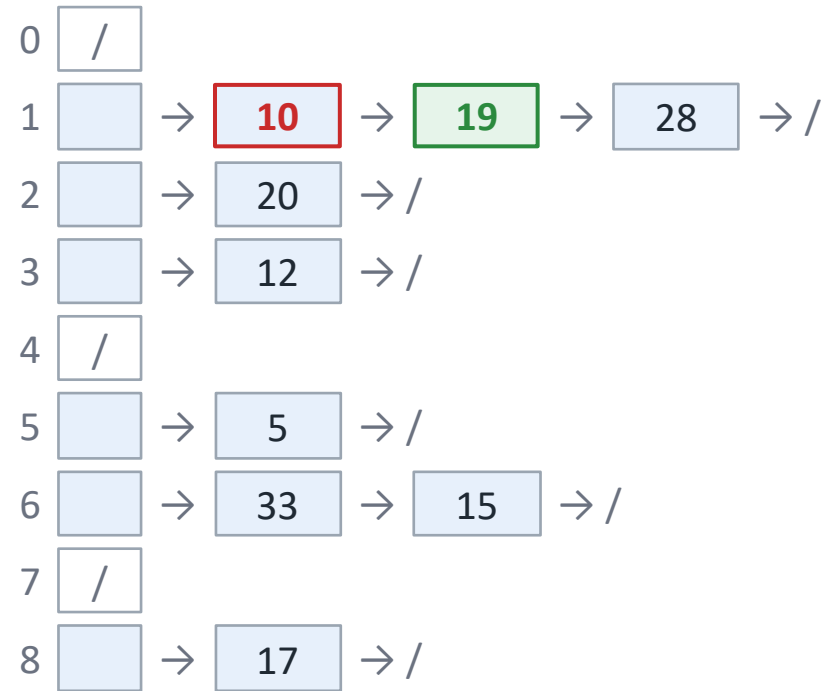
Done: 9 keys in 9 slots. Longest list: slot 1, with **3 keys**.



Searching the Chained Table

- **Search 19**: slot 1; compare 10, then **19 found**.
- **Search 28**: slot 1; compare 10, 19, 28.
- **Search 11**: slot 2; compare 20; **not found**.

Cost = length of **one** list.



The Load Factor

The average number of keys per slot:

$$\alpha = \frac{n}{m} = \frac{\text{number of keys}}{\text{number of slots}}$$

Simple uniform hashing. assume each key is equally likely to go to any slot. Then each list has **α keys** on average.

Simple Uniform Hashing

Assumption. every key is equally likely to hash to any of the m slots, independently of where the other keys went.

$$\Pr [h(k) = j] = \frac{1}{m} \text{ for every slot } j, \text{ independently for different keys}$$

- It models h as if it were a random function: good enough to predict average behaviour.
- It is an **assumption** about the keys and h , not a theorem: for any fixed h , some key sets break it.
- Open addressing uses a stronger version, **uniform hashing**: every probe sequence (permutation) is equally likely.

Lemma: Expected List Length

Lemma. under simple uniform hashing, every list has $\alpha = n/m$ keys on average.

Fix a slot j and write its length as a sum of indicators, one per key:

$$n_j = \sum_{i=1}^n I_i, \quad I_i = \begin{cases} 1 & h(k_i) = j \\ 0 & \text{otherwise} \end{cases} \quad \mathbb{E}[n_j] = \sum_{i=1}^n \Pr[h(k_i) = j] = \frac{n}{m} = \alpha$$

Lemma: Two Keys Collide with Probability $1/m$

For two different keys k and l , sum over the slot they might share and use independence:

$$\Pr [h(k) = h(l)] = \sum_{j=0}^{m-1} \Pr [h(k) = j] \cdot \Pr [h(l) = j] = m \cdot \frac{1}{m^2} = \frac{1}{m}$$

By linearity of expectation over all pairs of keys:

$$\mathbb{E}[\# \text{ colliding pairs}] = \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2m}$$

Example. $n = 1,000$ keys in $m = 1,000$ slots: about **500 colliding pairs** on average, even though $\alpha = 1$.

How Many Slots Stay Empty?

Slot j stays empty if each of the n keys independently misses it (probability $1 - 1/m$ each):

$$\Pr[\text{slot } j \text{ empty}] = \left(1 - \frac{1}{m}\right)^n \quad \mathbb{E}[\# \text{ empty slots}] = m \left(1 - \frac{1}{m}\right)^n \approx m e^{-\alpha}$$

using $1 - 1/m \approx e^{-1/m}$ for large m . Summing indicators over the m slots gives the expected count.

Load factor α	0.5	1	2
Expected empty slots	$\approx 60.7\%$	$\approx 36.8\%$	$\approx 13.5\%$

Proof: Collisions Come Early

Claim. with n keys in m slots, the chance of **no collision at all** is at most $e^{-n(n-1)/(2m)}$.

Insert keys one at a time: key $i + 1$ must miss the i slots already used, with probability $1 - i/m$. Then use $1 - x \leq e^{-x}$:

$$\Pr [\text{no collision}] = \prod_{i=0}^{n-1} \left(1 - \frac{i}{m}\right) \leq \prod_{i=0}^{n-1} e^{-i/m} = e^{-n(n-1)/(2m)}$$

Birthday paradox. for $m = 365$ and $n = 23$, the bound is $e^{-253/365} \approx 0.50$, so a shared birthday has probability about **1/2** (exactly 50.7%). In general collisions appear once $n \approx \sqrt{m}$.

Search Time with Chaining

Theorem. with simple uniform hashing, a search takes $\Theta(1 + \alpha)$ time on average: 1 to compute $h(k)$, α to scan the list.

Consequence. keep m proportional to n . Then α is a constant and **every operation takes $O(1)$ average time.**

Proof: Unsuccessful Search with Chaining

Theorem. with simple uniform hashing, an unsuccessful search takes $\Theta(1 + \alpha)$ time on average.

Let n_j be the length of list $T[j]$, so $n_0 + \dots + n_{m-1} = n$ and $E[n_j] = n/m$. A key k not in the table is equally likely to hash to any slot, and the search scans the whole list:

$$\mathbb{E}[n_{h(k)}] = \sum_{j=0}^{m-1} \Pr[h(k) = j] \cdot \mathbb{E}[n_j] = \sum_{j=0}^{m-1} \frac{1}{m} \cdot \frac{n}{m} = \frac{n}{m} = \alpha$$

Adding $O(1)$ to compute $h(k)$ gives $\Theta(1 + \alpha)$.

Proof: Successful Search with Chaining

Assume the key searched for is equally likely to be any of the n stored keys $k_1, \dots, k_n$, inserted in that order. New keys go to the head of a list, so searching for k_i examines k_i plus the keys inserted **after** it into the same list.

$$X_{ij} = \begin{cases} 1 & \text{if } h(k_i) = h(k_j) \\ 0 & \text{otherwise} \end{cases} \quad \mathbb{E}[X_{ij}] = \frac{1}{m}$$

$$\begin{aligned} \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij}\right)\right] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m}\right) \\ &= 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \end{aligned}$$

So a successful search also takes $\Theta(1 + \alpha)$ time.

Chaining in C

```
typedef struct Node { int key; struct Node *next; } Node;
Node **T; int M;                                // M buckets, calloc'd, all NULL

void chained_insert(int key) {                    // insert at head: O(1)
    Node *x = malloc(sizeof(Node));
    x->key = key;
    x->next = T[h(key)];
    T[h(key)] = x;
}
Node *chained_search(int key) {                  // walk one list
    for (Node *x = T[h(key)]; x; x = x->next)
        if (x->key == key) return x;
    return NULL;
}
```

Rehashing in C

```
void rehash(void) {
    int old_m = M;
    Node **old = T;
    M = 2 * old_m;
    T = calloc(M, sizeof(Node *));
    for (int i = 0; i < old_m; i++) {
        Node *x = old[i];
        while (x != NULL) {
            Node *next = x->next;
            x->next = T[h(x->key)];
            T[h(x->key)] = x;
            x = next;
        }
    }
    free(old);
}
```

// double the number of buckets

// h(key) now uses the new M

// move each node to its new list

Hash Functions

What Makes a Good Hash Function?

- **Uniform**: keys spread evenly over the slots.
- **Deterministic**: same key, same slot.
- **Fast**: a few arithmetic operations.
- **Uses the whole key**: patterns in keys do not cause collisions.

Keys as numbers. the string "pt" becomes $(112 \times 128) + 116 = 14452$, reading ASCII codes as base-128 digits.

The Division Method

$h(k) = k \bmod m$: the remainder when k is divided by m .

Examples with $m = 12$:

- $k = 100 \rightarrow 100 \bmod 12 = 4$
- $k = 25 \rightarrow 25 \bmod 12 = 1$
- $k = 36 \rightarrow 36 \bmod 12 = 0$

Choosing m for the Division Method

Good. a **prime** not too close to a power of 2. Example: about 2,000 strings, $m = 701$.

Bad. $m = 2^p$. Then $h(k)$ is just the **lowest p bits** of k ; the other bits are ignored.

Also bad. $m = 10^d$ for decimal keys: only the last d digits matter.

The Multiplication Method

$$h(k) = \lfloor m (kA \bmod 1) \rfloor$$

- Multiply k by a constant A between 0 and 1.
- Keep only the **fractional part**.
- Multiply by m and round down.

Universal Hashing

Why Choose the Hash Function at Random?

Danger. for **any fixed** hash function, someone can pick keys that all land in one slot. Search then takes $\Theta(n)$.

Fix. choose the hash function **at random** from a family when the table is created. Then no input is bad on average.

Every Fixed Hash Function Has a Bad Input

Claim. if the universe has at least $N \cdot M$ keys, then for any hash function some N keys all land in the same bucket.

Why. pigeonhole: if every bucket received at most $N - 1$ keys of the universe, the universe would have fewer than $N \cdot M$ keys.

So a guarantee for **every** input must come from choosing h at random.

Universal Hashing

Definition. a family H of hash functions is **universal** if, for any two different keys $x \neq y$, a random h from H makes them collide with probability at most $1/M$.

Theorem. for any set of N keys, the expected number of keys that collide with a given key is less than N/M .

Universal Family: $(ak + b) \bmod p \bmod m$

Pick a prime p larger than every key, then choose a and b at random:

$$h_{a,b}(k) = ((a k + b) \bmod p) \bmod m$$

Example, $p = 17$, $m = 6$: $h_{3,4}(8) = (28 \bmod 17) \bmod 6 = 5$

k	0	1	2	3	4	5	6	7	8	9	10
$h_{3,4}(k)$	4	1	4	1	4	2	5	2	5	2	0
$h_{5,2}(k)$	2	1	0	0	5	4	3	3	2	1	1

Any two different keys collide with probability at most $1/m$.

Universal Family: Dot Product mod a Prime

Split a key into pieces $x = (x_1, \dots, x_k)$, each less than a prime M . Pick $a_1, \dots, a_k$ at random:

$$h_a(x) = (a_1x_1 + a_2x_2 + \dots + a_kx_k) \bmod M$$

Example. $M = 7, a = (3, 5, 1)$: $h(2, 4, 6) = (6 + 20 + 6) \bmod 7 = 32 \bmod 7 = 4$.

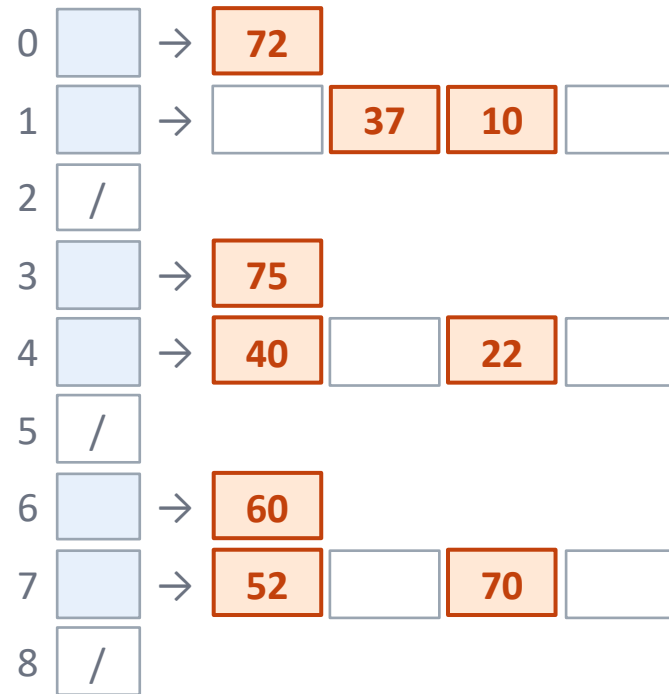
Why universal. if x and y differ in piece i , a collision forces a_i to one specific value mod M : probability exactly $1/M$.

Perfect Hashing

Perfect Hashing

- For a **fixed** set of keys, search in **$O(1)$ worst case**.
- **Level 1**: hash the n keys into n slots.
- **Level 2**: a slot with n_j keys gets its own table of size n_j^2 , with a hash function that has **no collisions**.
- Total space is still $O(n)$ on average.

Perfect Hashing: Example



- Keys 10, 22, 37, 40, 52, 60, 70, 72, 75.
- Level 1: $k \bmod 9$.
- Slots 1, 4, 7 hold 2 keys, so they get **size-4 tables**, using $k \bmod 4$.
- Search: two hash computations.

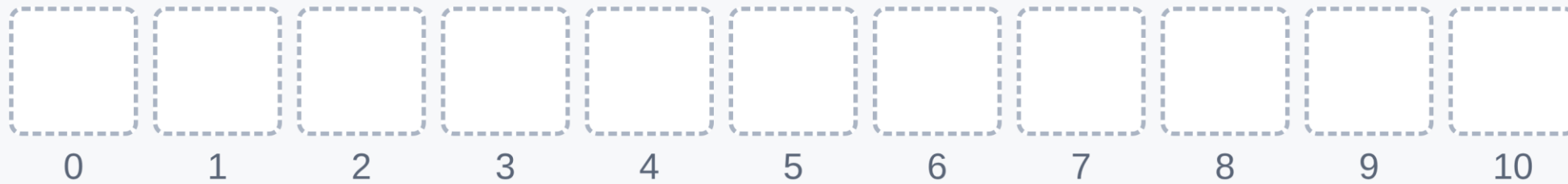
References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C. (2009). *Introduction to Algorithms*, 3rd ed. MIT Press.
- Woodruff, D. (2023). Lecture 5: Hashing. CMU 15-451, Spring 2023.
- Stanford CS106B (Summer 2020). Lecture 27: Hashing.
- MIT 6.006 (Fall 2010). Lecture 6: Hashing II (rolling hash, longest common substring).
- Knuth, D. E. (1998). *The Art of Computer Programming*, Vol. 3. Addison-Wesley.
- Fredman, M. L., Komlós, J. & Szemerédi, E. (1984). Storing a sparse table with $O(1)$ worst case access time. *JACM*.

Hashing Quiz

QUESTION 1 · NUMERICAL ANSWER

Linear probing with $m = 11$ and $h(k, i) = (k \bmod 11 + i) \bmod 11$. The table starts empty:



Keys 20, 31, 42, 9, 53 are inserted in this order. What is the total number of probes, counting the final empty slot of each insertion?

QUESTION 2 · NUMERICAL ANSWER

Quadratic probing with $m = 11$ and $h(k, i) = (k \bmod 11 + i + 3i^2) \bmod 11$. After inserting 11, 22, 33 the table is:

11			33	22						
0	1	2	3	4	5	6	7	8	9	10

Suppose the key is the last two digits of your roll number. Which slot will the key occupy?

QUESTION 3 · NUMERICAL ANSWER

Double hashing with $m = 11$, $h1(k) = k \bmod 11$ and $h2(k) = 1 + (k \bmod 10)$. After inserting 14, 25, 36 the table is:

			14						25	36
0	1	2	3	4	5	6	7	8	9	10

Key 47 is inserted next. In which slot does it land?

QUESTION 4 · MULTIPLE CHOICE

In open addressing, why is a deleted slot marked **DELETED** instead of being reset to NIL?

A To save memory

B So that later insertions skip the slot

C So a search does not stop early and miss keys further along the probe sequence

D So the load factor stays constant

QUESTION 5 · MULTIPLE CHOICE

In two-level perfect hashing (FKS), slot j of the primary table receives n_j keys, and its secondary table is sized to make collisions unlikely.

If slot j receives $n_j = 4$ keys, what is the size of its secondary table?

A 4

B 8

C 16

D 64

QUESTION 6 · MULTIPLE CHOICE

20 keys are inserted into a hash table with $m = 20$ slots. Assuming simple uniform hashing, what is the expected number of empty slots?

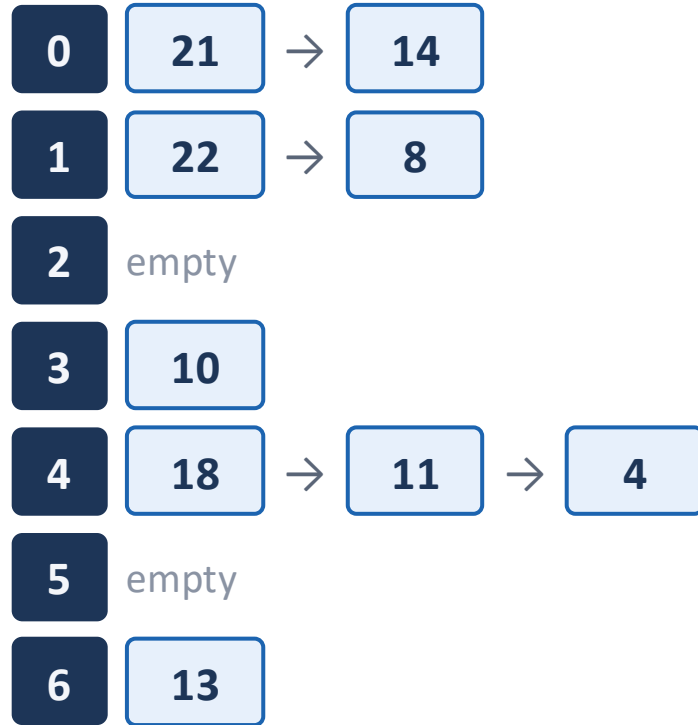
A $(19/20)^{20}$

B $20 \cdot (1 - (19/20)^{20})$

C $20 \cdot (19/20)^{20}$

D $20 \cdot (1/20)^{20}$

QUESTION 7 · NUMERICAL ANSWER



Consider collision resolution with chaining.
 $m = 7$ and $h(k) = k \bmod 7$.

New keys are inserted at the head of their chain.

Keys 43 and then 29 are inserted. How many key comparisons does a successful search for 8 then make?

QUESTION 8 · MULTIPLE CHOICE

A double-hashing table has $m = 16$ slots. Which secondary hash function guarantees that every probe sequence visits all 16 slots?

A $h_2(k) = k \bmod 16$

B $h_2(k) = 1 + (k \bmod 15)$

C $h_2(k) = 2 \cdot (k \bmod 8) + 1$

D $h_2(k) = 2 + (k \bmod 14)$